Confidential mode: what a Routstr node can and cannot see

The short version. In confidential mode, your routstrd talks to the AI provider (for example Venice) through the Routstr node. The node only relays encrypted traffic and adds its own API key. It cannot read your prompts or the answers, and it cannot change them. Billing is based on the usage figures the provider itself reports. You do not need new apps or accounts: it is a software upgrade of routstrd and of the node, switched on per request.

What you trust, and for what

Party	You trust it for	You no longer trust it for
The provider (e.g. Venice)	Running the model you asked for, and handling your data per its policy. It sees your prompt, as it always has.	
The Routstr node	Business honesty: taking your payment, relaying traffic, refunding unused money. Reputation covers this, as it does today.	Reading your prompts and answers, changing them, swapping the model, inflating the bill.
Your routstrd	Everything: it holds the encryption keys. It runs on your machine.	

The assumption behind this design: **nodes resell reputable providers, and you should not have to trust the node much.** Confidential mode moves most of the trust you used to place in the node onto cryptography. The part that remains (refunds, uptime) is ordinary business trust, which node reputation already handles.

The three common objections

1. "Can the node change my tool calls, system prompt or messages?" No. Your request body (messages, system prompt, tools, tool results) is encrypted with keys only your routstrd has. If the node changed even one byte, the provider would reject it as tampered. The same holds for the answer, including tool calls the model makes: your routstrd checks every piece against the provider's encryption, so a modified answer is detected and discarded. The node writes exactly one thing, the request header with its API key. A zero-knowledge proof shows your routstrd that this header matches a published template. (The node could still write a different header of the same length, for example its other account. It cannot use that to touch your content.)

2. "Can the node change the model, say to a cheaper one, or bill me for a dearer one?" No. The model name and the token limits sit at the end of your encrypted request, where they override anything earlier, and the node cannot alter encrypted bytes. The node holds back that last piece of the request until your routstrd has proven it names the agreed model and limits. Until then the provider has an incomplete request and does nothing. This also protects the node from a client swapping in a pricier model. Your routstrd also checks that it is really talking to the provider: the certificate must be valid and the hostname must be on your trusted list (wildcards allowed). The bill is computed from the provider's own usage report, which your routstrd proves to the node and checks against its own copy, and you receive a signed receipt. If the provider returns an error (for example "overloaded"), it verifiably costs you nothing. What weights the provider actually runs is the provider's honesty, which is why you pick reputable ones.

3. "Can the Routstr node read my prompts and responses?" No. It only ever handles ciphertext; the keys never leave your machine. What it can still see: which provider and model you use, when, how long your request is, and the size of each streamed chunk. Chunk sizes can leak some information about an answer (research has partially reconstructed responses from them), but this is far less than today, where the node sees everything in plain text.

What it costs, and the limits

- **Speed:** about +1 s to the first token in our tests (+0.3 s with pre-warmed sessions, not yet in routstrd); streaming then follows, a little slower than direct.
- **Money:** paid exactly like your other requests to that node, from the same balance. The node holds back the maximum and charges only the usage the provider reported. If the provider never ran your request, or answered with an error, nothing is charged.
- **Providers and models:** the provider must support standard TLS 1.3 behaviour that Venice has today; other providers need checking. Nodes currently offer text models in this mode.
- **Not covered:** the provider still sees your prompt; the node can still refuse service or keep money after a failed session (reputation is your protection there, as today).
- **No special hardware:** no enclaves, nothing to re-attest after updates. Just standard TLS and zero-knowledge proofs, running in the routstrd you already use.